

Angular Routing

1. What Is Angular Routing?

Angular Routing enables navigation between different views (pages) in a **Single Page Application (SPA)** without reloading the browser.

Instead of loading a new HTML page:

- Angular **changes the URL**
- Matches it against a **route configuration**
- Dynamically loads the **associated component**

This is handled by **@angular/router**.

2. Why Routing Is Required

Without routing:

- Entire UI logic stays in one component
- No meaningful URLs
- Poor UX and SEO

With routing:

- Clean URLs (/products, /orders/10)
 - Lazy loading for performance
 - Browser navigation support (Back/Forward)
 - Modular architecture
-

3. High-Level Routing Flow

Flow:

1. User clicks a link or enters URL
 2. Angular Router parses the URL
 3. Route is matched
 4. Guards (if any) execute
 5. Component is loaded inside <router-outlet>
-

4. Router Setup in Angular 20+ (Standalone Way)

Angular 20+ **does NOT** require NgModules.

4.1 Install Router (already included)

```
ng new my-app --routing
```

4.2 Define Routes

app.routes.ts

```
import { Routes } from '@angular/router';

import { HomeComponent } from './home/home.component';
import { ProductsComponent } from './products/products.component';

export const routes: Routes = [

  { path: '', component: HomeComponent },

  { path: 'products', component: ProductsComponent }
```

```
];
```

4.3 Register Router

main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';  
import { provideRouter } from '@angular/router';  
import { AppComponent } from './app/app.component';  
import { routes } from './app/app.routes';
```

```
bootstrapApplication(AppComponent, {  
  providers: [  
    provideRouter(routes)  
  ]  
});
```

5. Router Outlet – Where Components Render

app.component.html

```
<nav>  
  <a routerLink="/">Home</a>  
  <a routerLink="/products">Products</a>  
</nav>
```

```
<router-outlet></router-outlet>
```

- <router-outlet> is a **placeholder**
 - Routed components render here
-

6. RouterLink vs href

Bad Practice

```
<a href="/products">Products</a>
```

Reloads the page.

Correct

```
<a routerLink="/products">Products</a>
```

SPA navigation, no reload.

7. Route Parameters (Dynamic Routing)

Use Case

```
/products/101
```

7.1 Route Definition

```
{ path: 'products/:id', component: ProductDetailsComponent }
```

7.2 Reading Route Parameter

```
import { ActivatedRoute } from '@angular/router';
```

```
constructor(private route: ActivatedRoute) {}
```

```
ngOnInit() {  
  const id = this.route.snapshot.paramMap.get('id');  
}
```

8. Query Parameters

URL

```
/products?category=electronics&page=2
```

Read Query Params

```
this.route.queryParams.subscribe(params => {  
  console.log(params['category']);  
});
```

9. Programmatic Navigation

```
import { Router } from '@angular/router';
```

```
constructor(private router: Router) {}
```

```
goToProducts() {  
  this.router.navigate(['/products']);  
}
```

With params:

```
this.router.navigate(['/products', 101]);
```

10. Route Guards (Security & Access Control)

Guards control **whether navigation is allowed**.

10.1 Types of Guards

Guard	Purpose
CanActivate	Protect route
CanDeactivate	Warn before leaving
CanLoad	Prevent lazy module loading

Guard	Purpose
Resolve	Pre-fetch data

10.2 Example: Auth Guard

```
import { CanActivateFn } from '@angular/router';

export const authGuard: CanActivateFn = () => {
  return localStorage.getItem('token') !== null;
};
```

10.3 Apply Guard

```
{
  path: 'admin',
  component: AdminComponent,
  canActivate: [authGuard]
}
```

11. Lazy Loading (Performance Optimization)

Loads feature routes **only when needed**.

11.1 Lazy Route

```
{
  path: 'orders',
  loadComponent: () =>
    import('./orders/orders.component')
    .then(m => m.OrdersComponent)
```

```
}
```

Benefits:

- Faster initial load
 - Smaller bundle size
 - Scalable architecture
-

12. Child Routes (Nested Routing)

Example URL

/admin/users

/admin/settings

Route Configuration

```
{  
  path: 'admin',  
  component: AdminComponent,  
  children: [  
    { path: 'users', component: UsersComponent },  
    { path: 'settings', component: SettingsComponent }  
  ]  
}
```

Admin Template

```
<h2>Admin</h2>
```

```
<router-outlet></router-outlet>
```

13. Wildcard & Redirect Routes

Redirect

```
{ path: '', redirectTo: 'home', pathMatch: 'full' }
```

404 Page

```
{ path: '**', component: NotFoundComponent }
```

14. Router Events (Advanced)

Track navigation lifecycle.

```
this.router.events.subscribe(event => {  
  if (event instanceof NavigationEnd) {  
    console.log('Navigation complete');  
  }  
});
```

15. Typical Enterprise Routing Structure

src/app

└─ app.routes.ts

└─ auth/

| └─ auth.routes.ts

└─ products/

| └─ products.routes.ts

└─ admin/

└─ admin.routes.ts

Each feature owns its routing → **clean & scalable**

16. Routing Best Practices (Angular 20+)

- ✓ Use standalone routing
 - ✓ Prefer lazy loading
 - ✓ Guard sensitive routes
 - ✓ Keep route files feature-based
 - ✓ Avoid logic in components; delegate to services
-

17. Summary

Feature	Angular 20+
SPA navigation	Yes
Standalone routing	Yes
Lazy loading	Yes
Guards & security	Yes
Nested routes	Yes
Enterprise-ready	Yes
